

Query Optimization

Tune up

CPE 241 RDBMS
Week 9

Agenda

Relational calculus

Relational algebra

Query optimization

Database tune up

Relational Calculus

Based on **predicate calculus** dealing with first order logic

Non-procedural query language

Focus on **what** data to retrieve (no execution step)

Foundation of SQL (identify what you want without how to get it)

Has two types

Tuple Relational Calculus (TRC) – Queries are expressed in terms of **tuples** (rows).

Domain Relational Calculus (DRC) – Queries are expressed in terms of **domains** (column values).

Difference Between TRC and DRC

Aspect	TRC (Tuple Relational Calculus)	DRC (Domain Relational Calculus)
Variables	Tuple variables	Domain (attribute) variables
Refers to	Entire rows (tuples)	Individual column values
Focus	"Which tuples satisfy the condition?"	"Which values satisfy the condition?"
Notation	<code>t.Attribute</code>	Attribute variables directly
Closer to	Object / record view	Mathematical logic view

Queries: TRC

Relational Calculus represents queries using **mathematical logic tuple variables** that range over relations.

Tuple Relational Calculus (TRC)

tuple variables that range over relations.

$\{ t \mid P(t) \} \leftarrow$ this is a formula

- **t** is a tuple variable.
- **P(t)** is a **predicate (condition)** that defines the required properties of the tuples in the result.

Query: Retrieve employees with a salary greater than 50,000.

Employee(id, name, department, salary)

$\{ t \mid t \in \text{Employee} \wedge t.\text{Salary} > 50000 \}$

Explanation:

- **t** is a tuple variable that represents a row in the **Employee** relation.
- The condition $t.\text{Salary} > 50000$ filters only those employees whose salary exceeds 50,000.
- The result is a set of tuples matching the condition.

Queries: DRC

Domain Relational Calculus (DRC)

domain variables that refer to column values rather than entire tuples.

$$\{(d_1, d_2, \dots, d_n) \mid P(d_1, d_2, \dots, d_n)\}$$

- $d_1, d_2, \dots, d_n$ are domain variables representing specific columns.
- $P(d_1, d_2, \dots, d_n)$ is a predicate that defines the conditions on these columns.

Query: Retrieve employee names where salary is greater than 50,000.

$$\{ (\text{Name}) \mid \exists \text{Salary} (\text{Employee}(\text{Name}, \text{Salary}) \wedge \text{Salary} > 50000) \}$$

Explanation:

- The result set consists of only the **Name** column.
- The existential quantifier $\exists \text{Salary}$ ensures that the tuple exists in the **Employee** relation.
- The condition $\text{Salary} > 50000$ filters the records.

Formulas in Tuple Relational Calculus

Atomic Formulas

Atomic formulas are the **basic building blocks** of TRC and include:

1. **Tuple Membership:**
 $t \in R \rightarrow$ The tuple t belongs to the relation R
2. **Attribute Conditions:**
 $t.A=c \rightarrow$ The value of attribute A in tuple t is equal to a constant c .

Complex Formulas

Complex formulas are constructed using **logical operators** and **quantifiers**:

Negation ($\neg$): $\neg P(t)$ means "not $P(t)$ ".

Conjunction ($\wedge$): $P(t) \wedge Q(t)$ means "both $P(t)$ and $Q(t)$ must be true".

Disjunction ($\vee$): $P(t) \vee Q(t)$ means "either $P(t)$ or $Q(t)$ must be true".

Implication ($\Rightarrow$): $P(t) \Rightarrow Q(t)$ means "if $P(t)$ is true, then $Q(t)$ must also be true".

Universal Quantifier ($\forall$): Ensures a condition holds for **all** tuples in a relation.

Existential Quantifier ($\exists$): Ensures a condition holds for **at least one** tuple in a relation.

Quantifiers: Existential

Existential Quantifier ($\exists$) – "For Some"

The **existential quantifier** ($\exists$) states that **there exists at least one tuple** satisfying the given condition.

Example 1: Find Employees Working in the HR Department

$$\{t \mid t \in \text{Employee} \wedge \exists d(d \in \text{Department} \wedge d.\text{DeptName} = \text{'HR'} \wedge t.\text{DeptID} = d.\text{DeptID})\}$$

Explanation:

Selects all employees where **at least one** tuple d exists in the **Department** relation such that:

- The department name is **'HR'**.

- The employee's **DeptID** matches the department's **DeptID**.

Quantifiers Universal

Universal Quantifier ($\forall$) – "For All"

The **universal quantifier** ($\forall$) states that **a condition must be true for all tuples** in a relation.

Example 2: Find Employees Who Earn the Highest Salary

$$\{t \mid t \in \text{Employee} \wedge \forall s (s \in \text{Employee} \Rightarrow t.\text{Salary} \geq s.\text{Salary})\}$$

Explanation:

- Selects all employees t whose salary is **greater than or equal to every other** employee's salary.
- Ensures that **for every** employee s , the condition $t.\text{Salary} \geq s.\text{Salary}$ holds.

Operators in Tuple Relational Calculus

Comparison Operators

Operator	Meaning
=	Equal to
≠	Not equal to
>	Greater than
<	Less than
≥	Greater than or equal to
≤	Less than or equal to

Logical Operators

Operator	Meaning
¬	NOT
∧	AND
∨	OR
⇒	IMPLIES

More Examples of Tuple Relational Calculus (TRC)

Example 1: Retrieve Employees Working in the 'HR' Department

$$\{t \mid t \in \text{Employee} \wedge \exists d(d \in \text{Department} \wedge d.\text{DeptName} = \text{'HR'} \wedge t.\text{DeptID} = d.\text{DeptID})\}$$

Explanation:

- We retrieve all tuples t from the **Employee** relation.
- The condition ensures that $t.\text{DeptID}$ matches a **DeptID** from the **Department** relation where **DeptName** is 'HR'.
- The result is a set of employees working in the HR department.

Example 2: Retrieve Names of Employees Who Earn More than 60,000

$$\{t.\text{Name} \mid t \in \text{Employee} \wedge t.\text{Salary} > 60000\}$$

Explanation:

- This query retrieves only the **Name** attribute of employees.
- The condition ensures that only employees with a salary greater than 60,000 are included.

Example 3: Find Employees Who Work in the Same Department as 'Alice'

$$\{t \mid t \in \text{Employee} \wedge \exists s(s \in \text{Employee} \wedge s.\text{Name} = \text{'Alice'} \wedge t.\text{DeptID} = s.\text{DeptID})\}$$

Explanation:

- Retrieves all employees who share the same **DeptID** as 'Alice'.
- Uses the **Employee** relation twice: **s** represents Alice, and **t** represents employees in her department.

Example 4: Retrieve Employees Who Earn the Highest Salary

$$\{t \mid t \in \text{Employee} \wedge \forall s(s \in \text{Employee} \Rightarrow t.\text{Salary} \geq s.\text{Salary})\}$$

Explanation:

- The condition ensures that **t** has a salary that is **greater than or equal to** all other employees' salaries.
- This results in selecting employees who have the highest salary in the company.

More Examples of Domain Relational Calculus (DRC)

Example 1: Retrieve Names of Employees Working in the 'HR' Department

$$\{(Name) \mid \exists ID, Salary, DeptID (Employee(Name, ID, Salary, DeptID) \wedge \exists DeptName (Department(DeptID, DeptName) \wedge DeptName = 'HR'))\}$$

Explanation:

- Retrieves **Name** from the **Employee** table.
- The condition ensures that the employee's **DeptID** matches a **DeptID** in the **Department** table where the department name is 'HR'.

Example 2: Retrieve Employee IDs and Names of Employees Earning More than 60,000

$$\{(ID, Name) \mid \exists Salary, DeptID (Employee(ID, Name, Salary, DeptID) \wedge Salary > 60000)\}$$

Explanation:

- The result contains **ID** and **Name** of employees who have a **Salary** greater than 60,000.
- **Salary** and **DeptID** are used only in the condition.

Example 3: Retrieve Employees Who Work in the Same Department as 'Alice'

$\{(Name) \mid \exists ID, Salary, DeptID (Employee(Name, ID, Salary, DeptID) \wedge \exists AID, ASalary (Employee('Alice', AID, ASalary, DeptID)))\}$

Explanation:

- Retrieves names of employees.
- Ensures that the employee's **DeptID** is the same as Alice's **DeptID**.

Example 4: Retrieve Employees Who Earn the Highest Salary

$\{(Name) \mid \exists Salary, ID, DeptID (Employee(Name, ID, Salary, DeptID) \wedge \forall S, EID, EDeptID (Employee(EID, S, EDeptID) \Rightarrow Salary \geq S))\}$

Explanation:

- Retrieves names of employees with the **highest salary**.
- The universal quantifier ($\forall$) ensures that the selected employee's salary is greater than or equal to **every other** employee's salary.

Relational Algebra

procedural query language

how to retrieve it step by step

foundation for **query optimization, SQL query execution plans**

based on **Relational Algebra operations**.

Basic Operations in Relational Algebra

Selection (σ) – Filtering Rows

Projection (π) – Selecting Columns

Union ($\cup$) – Combining Relations

Set Difference ($-$) – Subtracting Relations

Cartesian Product ($\times$) – Combining All Pairs

Rename (ρ) - $\rho_x(R)$ save R to x

Selection (σ) & Project Operation (Π)

The **selection** operator **filters** tuples (rows) from a relation based on a given condition.

$$\sigma_{\text{condition}}(R)$$

The **projection** operator **retrieves specific attributes (columns)** from a relation.

$$\Pi_{\text{attribute1, attribute2, ..., attributeN}}(R)$$

$$\sigma_{\text{Salary} > 10000 \wedge \text{Dept} = \text{'HR'}}(\text{Employee})$$

$$\Pi_{\text{name, dept}}(\sigma_{\text{Salary} > 1000000}(\text{Employee}))$$

Union ($\cup$) & Set Difference ($-$)

The **union** operator **combines** two relations and removes duplicates.

$R1 \cup R2$

$\pi_{\text{Name}}(\sigma_{\text{Dept}='HR'}(\text{Employee})) \cup \pi_{\text{Name}}(\sigma_{\text{Dept}='Finance'}(\text{Employee}))$

The **set difference** operator **removes** tuples from one relation that appear in another.

$R1 - R2$

$\pi_{\text{Name}}(\sigma_{\text{Dept}='HR'}(\text{Employee})) - \pi_{\text{Name}}(\sigma_{\text{Dept}='Finance'}(\text{Employee}))$

- Both relations must have the **same number of attributes**.
- The attributes must be **of the same type**.

Cartesian Product ($\times$)

The **Cartesian product** (or **cross join**) **combines every tuple** from one relation with **every tuple** from another.

Employee $\times$ Department

Employee = { (A, HR), (B, S), (C, C) } Department = { (HR, Human Resource), (S, Sale), (C, Computer) }

A	HR	HR	Human Resource
A	HR	S	Sale
A	HR	C	Computer
B	S	HR	Human Resource
B	S	S	Sale
B	S	C	Computer
C	C	HR	Human Resource
C	C	S	Sale
C	C	C	Computer

Join

Join ($\bowtie$) – Combining Related Relations

$R1 \bowtie \text{condition} R2$

Natural Join ($\bowtie$) – Automatically joins on common attributes.

Theta Join ($\bowtie$ Condition) – Joins using specific conditions.

Equi-Join ($\bowtie A = B$) – Joins where attributes are equal.

Outer Join (Left, Right, Full) – Includes unmatched tuples.

Natural Join ($\bowtie$)

Automatically joins on common attributes.

$R1 \bowtie R2$

EmpID	Name	DeptID	DeptID	DeptName
101	Alice	1	1	HR
102	Bob	2	2	Finance
103	Charlie	1		

EmpID	Name	DeptID	DeptName
101	Alice	1	HR
102	Bob	2	Finance
103	Charlie	1	HR

Theta Join ($\bowtie_{\theta}$)

allows **any comparison condition** (e.g., $=$, $>$, $<$, $\geq$, $\leq$, $\neq$) between attributes from both relations.

$R1 \bowtie_{\text{condition}} R2$

$\text{Employee} \bowtie_{\text{Employee.DeptID=Department.DeptID} \wedge \text{Department.DeptID}=1} \text{Department}$

Can match **different attribute names**.

EmpID	Name	DeptID	DeptName
101	Alice	1	HR
103	Charlie	1	HR

Equi-Join ($\bowtie=$)

An **Equi-Join** is a **special case of Theta Join** where the **only condition** is an **equality (=) condition**.

$R1 \bowtie_{A=B} R2$

$\text{Employee} \bowtie_{\text{Employee.DeptID}=\text{Department.DeptID}} \text{Department}$

EmpID	Name	DeptID	DeptID	DeptName
101	Alice	1	1	HR
102	Bob	2	2	Finance
103	Charlie	1	1	HR

Left Outer Join ($\bowtie$)

all tuples from the left relation and matching tuples from the right

R1 $\bowtie_{\text{condition}}$ R2

		EmpID	Name	DeptID
		101	Alice	1
		102	Bob	2
1	HR	103	Charlie	1
2	Finance	104	David	3

EmpID	Name	DeptID	DeptName
101	Alice	1	HR
102	Bob	2	Finance
103	Charlie	1	HR
104	David	3	NULL

Right Outer Join ($\bowtie$)

all tuples from the right relation and matching tuples from the left

$R1 \bowtie_{\text{condition}} R2$

$\text{Employee} \bowtie_{\text{Employee.DeptID=Department.DeptID}} \text{Department}$

EmpID	Name	DeptID	DeptName
101	Alice	1	HR
102	Bob	2	Finance
103	Charlie	1	HR
NULL	NULL	NULL	Marketing

Full Outer Join ($\bowtie$)

all tuples from both relations.

$R1 \bowtie_{\text{condition}} R2$

$\text{Employee} \bowtie_{\text{Employee.DeptID=Department.DeptID}} \text{Department}$

EmpID	Name	DeptID	DeptName
101	Alice	1	HR
102	Bob	2	Finance
103	Charlie	1	HR
104	David	3	NULL
NULL	NULL	4	Marketing

Query Optimization

A **Query Execution Plan** is the **step-by-step strategy** the database engine uses to execute a given query. It provides a detailed breakdown of:

- **Access Methods** → How tables are accessed (scanning, indexing)
- **Join Strategies** → Join methods (nested loops, hash joins, merge joins)
- **Operators** → Sorting, Aggregation, and filtering steps
- **Cost Estimation** → The estimated cost of each operation

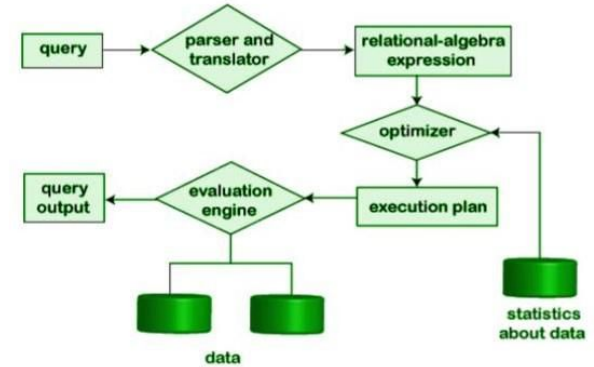
Database Management Systems (DBMS) generate **multiple possible execution plans** for a query and choose the one with the **lowest cost**.

SQL Query → Multiple Possible Plans → Lowest Estimated Cost

Query Execution Process

When a query is executed in a relational database, it goes through **several stages**:

1. **Parsing:**
SQL syntax and basic validity
 - If errors are found, execution stops.
2. **Binding** (Semantic Checking):
Tables, columns, types exist
 - Validates column types and constraints.
3. **Optimization:**
The **query optimizer** Generate and compare execution plans
 - It evaluates each plan and selects the **most efficient one**.
4. **Execution:**
Run chosen plan
5. **Result Output:**
Return results to user



Steps in query processing

[geeksforgeeks pipeline-in-query-processing-in-dbms/](https://www.geeksforgeeks.org/pipeline-in-query-processing-in-dbms/)

Components of a Query Execution Plan

Accessing Data: Table Access Methods

- **Full Table Scan:** The entire table is scanned.
- **Index Scan:** An index is used to locate matching rows.
- **Index Seek:** A more efficient version of an index scan that goes directly to relevant index entries.

Combining Tables: Join Methods

- **Nested Loop Join:** Iterates over one table and finds matches in the second table.
- **Hash Join:** Builds a hash table from one relation and probes it with the second relation.
- **Merge Join:** Sorts both tables and merges them in a sorted manner.

Ordering and Summarizing Data: Sorting and Aggregation

- **Sorting:** Ordered operations use sorting algorithms.
- **Aggregation:** Functions like `SUM()`, `COUNT()`, and `AVG()` require grouping.

Reducing Output Size: Filtering and Projection (Very Important!)

- **Selection (σ):** Filters rows based on conditions.
- **Projection (π):** Selects specific columns.

Query Optimization Techniques

1 Reduce Data Early

Predicate Pushdown (Apply Filters Early)

Use WHERE Instead of HAVING

Avoiding SELECT*

Optimizing Joins

2 Use Efficient Access Paths

Using Indexes

Partitioning Large Tables

3 Reduce Repeated Work

Caching Results

Reusing execution plans

 Analyze → Using Query Execution Plan

Predicate push down

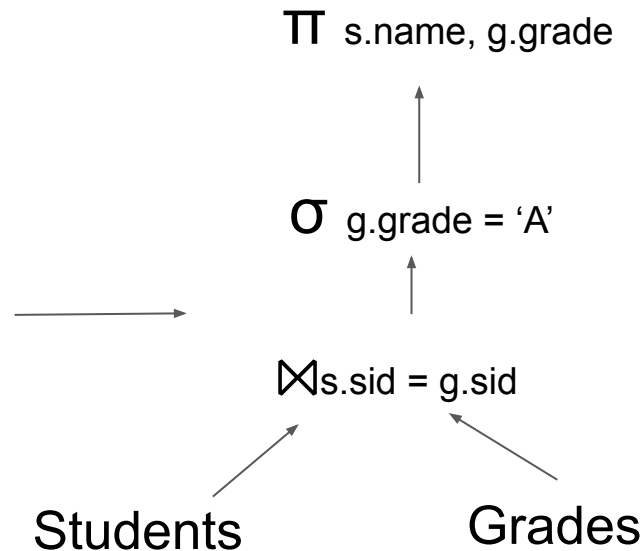
SELECT s.name, g.grade

FROM students as s, grades as g

WHERE s.sid = e.sid

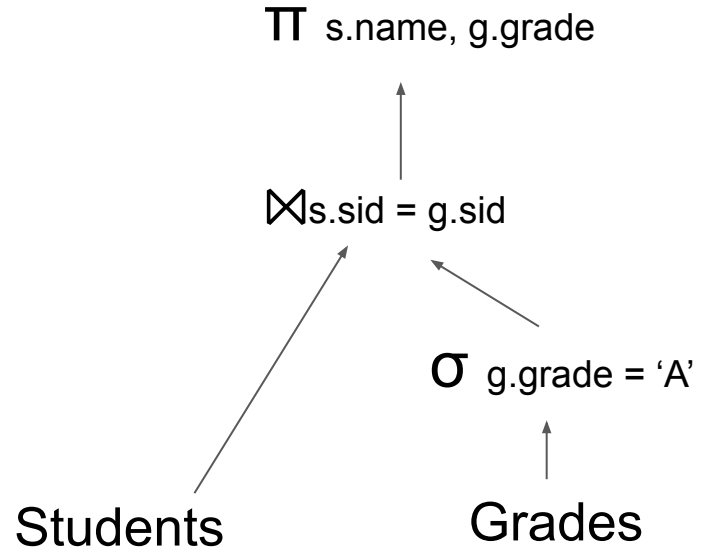
AND e.grade = 'A'

$\Pi_{s.name, g.grade}(\sigma_{g.grade = 'A'}(students \bowtie grades))$



Equivalent Relational Algebra

$\pi_{s.name, g.grade}(\text{students} \bowtie \sigma_{g.grade = 'A'}(\text{grades}))$



Projection Pushdown

You can also do projection pushdown

In case the students table has so many attributes that is not related in the evaluation and result.

Students(ID, Name, blah1, blah2, blah3,... blah100) $\rightarrow \Pi_{s.name, s.ID}(\text{Students})$

Using Indexes

Without index, DBMS do table scan

```
SELECT * FROM Employee WHERE Salary > 50000;
```

```
CREATE INDEX idx_employee_salary ON Employee (Salary);
```

Optimizing Joins

Ensure indexed columns

Join smaller tables first ← heuristic, maybe you can use statistics too

A 1000 records, B 400000 records, C 500 records.

$A \bowtie B \bowtie C$

$A \bowtie C \bowtie B$

$B \bowtie A \bowtie C$

$B \bowtie C \bowtie A$

$C \bowtie A \bowtie B$

$C \bowtie B \bowtie A$

Avoiding SELECT*

Of course, reduces data transfers

```
SELECT *  
FROM STUDENTS  
WHERE program = 'Computer Science';
```



```
SELECT student_id  
FROM STUDENTS  
WHERE program = 'Computer Science';
```



STUDENTS

student_id
first_name
last_name
date_of_birth
email
phone
address
national_id
program
year
gpa
advisor_id
registration_date
status

Use WHERE Instead of HAVING

HAVING filters **after** aggregation.

WHERE filters **before** aggregation (more efficient).

```
SELECT DeptID, AVG(Salary)
```

```
FROM Employee
```

```
WHERE Salary > 50000
```

```
GROUP BY DeptID;
```

Partitioning Large Tables

Horizon and Vertical

Too many rows → Vertical Partitioning

A-Z → A-H, I-R, S-Z

Too many columns → Horizontal Partitioning

Students → S1(ID, blah1-blah10), S2(ID,blah11-blah20),...

Using Query Execution Plan

Most databases allow you to analyze the query plan

```
EXPLAIN ANALYZE SELECT * FROM Employee WHERE Salary > 50000;
```

MySQL has Benchmark Tool

Indexing in Relational Databases

Motivation: The Problem Without Indexes

Consider a table with **millions of rows**:

STUDENT(student_id, name, program, year, gpa, ...)

Query:

```
SELECT student_id  
FROM STUDENT  
WHERE program = 'Computer Science';
```

Show more lines

 Without an index:

- The DBMS must **scan every row**
- High disk I/O
- Slow response time
- Poor scalability as data grows

What Is an Index?

An index is a data structure that allows the DBMS to quickly locate rows without scanning the entire table.

- Similar to an **index in a textbook**
- Stores **column values + pointers to rows**
- Enables fast **search, lookup, and filtering**

 Indexes are used automatically by the **query optimizer**

Indexes and Query Optimization (Key Connection)

- During optimization, the DBMS evaluates:
 - Table scan vs index access
 - Index scan vs index seek
- The optimizer chooses the **lowest-cost execution plan**

Example insight:

Using an index can change a query from **$O(n)$** scanning to **$O(\log n)$** lookup.

Indexes are especially useful for:

- **WHERE** conditions
- **JOIN** columns
- **ORDER BY**
- **GROUP BY**



This is why indexes appear in **query execution plans**

Indexes Are a Trade-Off

Indexes **improve read performance**, but they are **not free**:

-  Faster **SELECT**
-  Slower **INSERT, UPDATE, DELETE**
-  Extra storage required

Indexing is always a **performance trade-off**, managed by the optimizer and the designer.

Pros

- **Speeds up SELECT queries**
- **Reduces disk I/O**
- **Improves query performance** (especially for large tables)

Cons

- **Slower INSERT, UPDATE, DELETE operations** (because indexes must be updated).
- **Consumes additional disk space.**

Create an Index

Basic Syntax

```
CREATE INDEX index_name ON table_name (column_name);
```

Examples

```
CREATE INDEX idx_employee_name ON Employee(Name);
```

```
CREATE INDEX idx_employee_dept_salary ON Employee(DeptID, Salary);
```

Remove index

```
DROP INDEX idx_employee_name;
```

Types of Indexes

1. Primary Index (Clustered Index)
2. Secondary Index (Non-Clustered Index)
3. Unique Index
4. Full-Text Index
5. Partial Index
6. Bitmap Index
7. Covering Index
8. Hash Index

More tune up (single server environment)

De-normalization

- Reduce join

- Beware insert update delete

Precompute Aggregation

- Compute some statistics in advance

Temporary Tables for Query Optimization

- When **a query result needs to be reused** multiple times.

- When **data needs to be pre-processed before final aggregation**.

- When **queries involve expensive joins or subqueries**.

Materialized Views